Informe de aplicación de alumnos en Flutter

Sección A Gestión de alumnos

17 de octubre de 2026

La aplicación permite iniciar sesión, registrar alumnos y consultar sus notas con su estado de aprobación. Se ejecutó en Chrome y el registro de harold santiváñez, con nota 17, aparece en el listado. El informe presenta las cinco pantallas de la sección A y explica los bloques de código que intervienen en su funcionamiento.

1 Entorno y acceso a la sección A

Se utilizó Visual Studio Code en Windows, con Flutter y Dart. La aplicación se abrió en localhost:5200 mediante flutter run -d chrome --web-port 5200. Las pantallas y la navegación se encuentran en lib/main.dart; los modelos y el almacenamiento local, en lib/app_store.dart.



Figura 1. Pantalla de entrada. Se selecciona Alumnos para acceder a la sección A.

Inicio de la aplicación

Código	Comentario

lib/main.dart main
`WidgetsFlutterBinding.ensureInitialized();`
`runApp(LabApp(store: await AppStore.load()));`
	La primera instrucción prepara Flutter antes de acceder al almacenamiento. AppStore.load recupera los datos locales y los entrega a LabApp. Así, las pantallas utilizan el mismo estado de la aplicación. El inicio también incluye un manejo de errores y una opción de reintento si falla la lectura.

Los códigos del informe son fragmentos de los archivos implementados. Se muestran las partes relevantes y se ajustan los saltos de línea para facilitar su lectura. Las capturas se encuadran sobre el área de la aplicación.

2 Inicio de sesión de alumnos

La pantalla solicita correo y contraseña. El acceso de demostración utilizado fue docente@demo.com con la contraseña Demo1234. La contraseña se muestra oculta y el botón permite ingresar al menú cuando los datos son correctos.

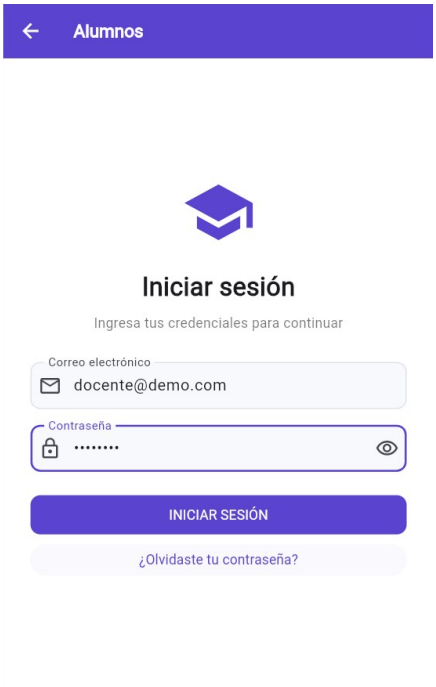


Figura 2. Inicio de sesión de alumnos con el correo ingresado y la contraseña oculta.

Validación y apertura del menú

Código	Comentario

lib/main.dart _LoginPageState._login
`if (!_form.currentState!.validate()) return;`
`if (!AppScope.of(context).login(`
`  widget.section, _email.text, _password.text)) {`
`  notice(context,`
`  'El correo o la contraseña no son correctos.');`
`  return;`
`}`
`ScaffoldMessenger.of(context)`
`  .hideCurrentSnackBar();`
`Navigator.of(context).pushReplacement(`
`  MaterialPageRoute<void>(`
`  builder: (_) =>`
`  MenuPage(section: widget.section)));`
	Primero se validan los campos del formulario. Después se comparan las credenciales con la cuenta de la sección seleccionada. Si no coinciden, aparece un mensaje y se mantiene el formulario. Cuando son válidas, pushReplacement abre el menú y reemplaza la ruta del inicio de sesión.

lib/app_store.dart AppStore.login
`bool login(Section section, String email,`
`  String password) =>`
`  email.trim().toLowerCase() ==`
`  profile(section).email.toLowerCase()`
`  && password == demoPassword;`
	La comparación del correo ignora espacios al inicio y al final, además de las mayúsculas. La contraseña debe coincidir exactamente. Se trata de un acceso local de demostración; esta implementación no consulta un servidor de autenticación.

3 Menú de alumnos

Después de iniciar sesión se presentan las opciones Registrar alumno, Listar alumnos, Perfil y Cerrar sesión. Cada opción dirige a una pantalla o acción específica.

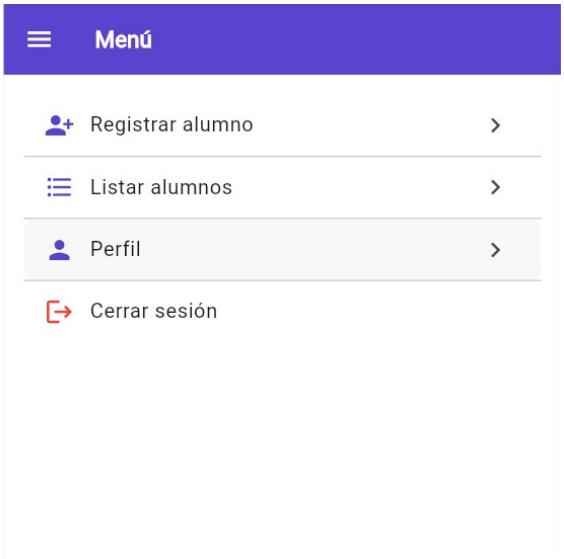


Figura 3. Menú principal de alumnos con sus cuatro opciones.

Navegación entre las pantallas

Código	Comentario

lib/main.dart openPage
`final navigator = Navigator.of(context);`
`if (fromDrawer) navigator.pop();`
`if (index == 3) {`
`  logout(context, section);`
`  return;`
`}`
`final Widget page;`
`if (index == 0) {`
`  page = RecordForm(section: section);`
`} else if (index == 1) {`
`  page = DataListPage(section: section);`
`} else {`
`  page = ProfilePage(section: section);`
`}`
`navigator.push(MaterialPageRoute<void>(`
`  builder: (_) => page));`
	El índice identifica la opción elegida. El valor 0 abre el formulario, el 1 abre el listado y el 2 muestra el perfil. El valor 3 cierra la sesión. Navigator.push agrega la pantalla al historial, lo que permite regresar. Si la opción se eligió desde el menú lateral, primero se cierra ese panel.

lib/main.dart logout
`Navigator.of(context).pushAndRemoveUntil(`
`  MaterialPageRoute<void>(`
`  builder: (_) => LoginPage(section: section)),`
`  (route) => route.isFirst);`
	El cierre de sesión regresa al formulario de acceso y elimina las pantallas internas del historial. La primera ruta se conserva para poder volver al selector inicial. Esta acción pertenece a la navegación local de la aplicación.

4 Registro de un alumno

El formulario contiene nombre, apellido y nota de 0 a 20. Se ingresaron los datos harold, santiváñez y 17. La captura muestra los campos antes de pulsar GUARDAR; el resultado del registro aparece en la figura 5.

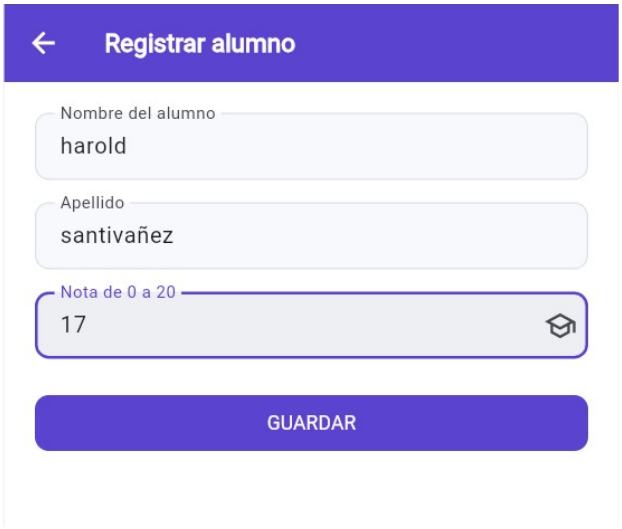


Figura 4. Formulario completado para registrar a harold santiváñez con nota 17.

Control de los datos y guardado

Código	Comentario

lib/main.dart validador de la nota
`final grade = readDecimal(value ?? '');`
`return grade == null || !grade.isFinite ||`
`  grade < 0 || grade > 20`
`? 'Ingresa una nota entre 0 y 20.'`
`: null;`
	La nota se convierte a un número decimal y se comprueba que tenga un valor válido dentro del intervalo permitido. Si el campo está vacío, no es numérico o queda fuera del rango, el formulario muestra el mensaje y evita guardar. El nombre y el apellido también son obligatorios.

lib/main.dart _RecordFormState._save
`await store.saveStudent(Student(`
`id: widget.student?.id ?? DateTime.now()`
`.microsecondsSinceEpoch.toString(),`
`name: _name.text.trim(),`
`surname: _surname.text.trim(),`
`grade: readDecimal(_grade.text)!,`
`));`
	Se construye un Student con los valores del formulario. Un alumno nuevo recibe un identificador basado en la fecha y hora; al editar se conserva el existente. trim elimina los espacios sobrantes. await espera a que AppStore termine el guardado antes de volver a la pantalla anterior.

lib/app_store.dart Student
`String get fullName => '$name $surname';`
`bool get approved => grade >= 11;`
	fullName une nombre y apellido para el listado. El estado se calcula a partir de la nota. En esta implementación se eligió 11 como nota mínima aprobatoria: desde 11 aparece Aprobado y con una nota menor aparece Desaprobado.

5 Listado y estado de los alumnos

El listado muestra cinco alumnos. El registro nuevo aparece al final con nota 17 y estado Aprobado. También se observa a María López con nota 09 y estado Desaprobado, de acuerdo con la condición utilizada en el código.

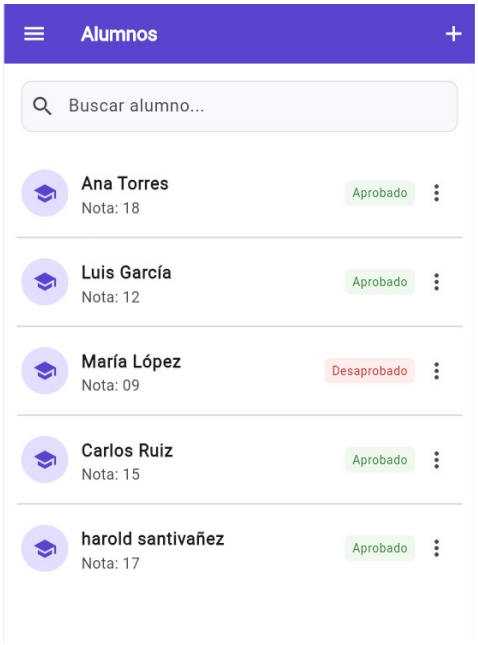


Figura 5. Listado actualizado con harold santiváñez y su nota 17.

Búsqueda y presentación de los registros

Código	Comentario

lib/main.dart _DataListPageState.build
`final students = store.students.where(`
`  (item) => item.fullName.toLowerCase()`
`  .contains(_query)).toList();`
 	
`onChanged: (value) => setState(() =>`
`  _query = value.trim().toLowerCase()),`
	El filtro busca coincidencias dentro del nombre completo. Tanto el texto del alumno como la consulta se comparan en minúsculas. setState actualiza la consulta cuando se escribe en el buscador y permite reconstruir el listado con los resultados correspondientes.

lib/main.dart elementos del listado
`title: Text(student.fullName,`
`  style: const TextStyle(`
`  fontWeight: FontWeight.w600)),`
`subtitle: Text(`
`  'Nota: ${gradeText(student.grade)}'),`
 	
`child: Text(student.approved`
`  ? 'Aprobado' : 'Desaprobado',`
`  style: TextStyle(`
`  color: color.shade800, fontSize: 11)),`
	Cada elemento presenta el nombre completo y la nota. gradeText aplica el formato de la nota, por eso el valor 9 se ve como 09. La etiqueta consulta approved para mostrar el estado. El código asigna verde a los aprobados y rojo a los desaprobados, como se aprecia en la captura.

6 Perfil del usuario

La pantalla presenta el nombre Harold Santiváñez, el correo docente@demo.com, el teléfono 912345678 y el rol Docente. Incluye el botón EDITAR PERFIL y un acceso para seleccionar una foto.

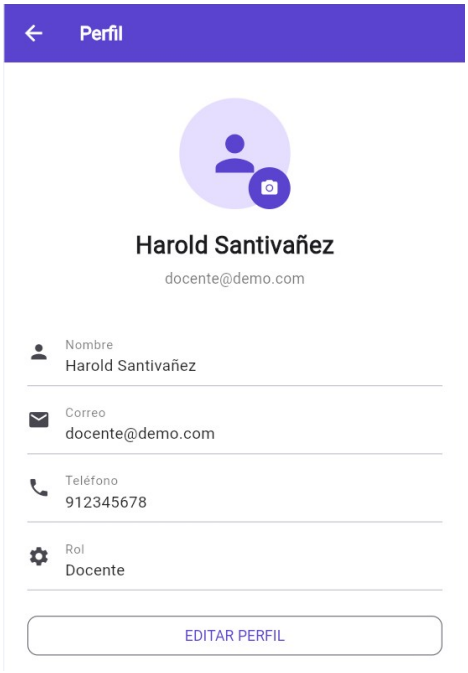


Figura 6. Perfil de la sección A con los datos actuales del usuario.

Lectura y actualización del perfil

Código	Comentario

lib/main.dart _ProfilePageState.build
`final user = AppScope.of(context)`
`  .profile(widget.section);`
	La pantalla obtiene el perfil guardado para la sección seleccionada. A partir de user se muestran el nombre, correo, teléfono, rol y foto. Los datos se leen del estado compartido, de modo que la interfaz puede presentar los cambios realizados mediante el formulario de edición.

lib/main.dart _EditProfilePageState._save
`await store.saveProfile(widget.section,`
`  UserProfile(`
`  name: _name.text.trim(),`
`  email: _email.text.trim().toLowerCase(),`
`  phone: _phone.text.trim(),`
`  role: current.role,`
`  photo: current.photo));`
`if (!mounted) return;`
`notice(context, 'Perfil actualizado.');`
`Navigator.of(context).pop();`
	El formulario crea un perfil con los datos editados y conserva el rol y la foto existentes. Cuando termina el guardado, mounted comprueba que la pantalla siga activa antes de utilizar su contexto. Se muestra un aviso y se regresa al perfil. Si se cambia el correo, AppStore utiliza ese nuevo correo en el siguiente acceso.

7 Almacenamiento y comprobación

AppStore administra los registros y el perfil. Para guardarlos utiliza SharedPreferencesAsync y JSON, un formato de texto que permite representar los datos del programa.

Código	Comentario

lib/app_store.dart Student.toJson
`Map<String, dynamic> toJson() => {`
`  'id': id,`
`  'name': name,`
`  'surname': surname,`
`  'grade': grade,`
`};`
	El método convierte los campos del alumno en un mapa. Esta estructura se incluye en el contenido que jsonEncode transforma en texto. Al cargar la aplicación, Student.fromJson reconstruye el objeto y convierte la nota nuevamente a double.

lib/app_store.dart fragmentos de _commit
`if (_prefs != null)`
`  await _prefs.setString(storageKey, data);`
`_students = nextStudents;`
 	
`notifyListeners();`
	data contiene los registros y perfiles codificados como JSON. El guardado termina antes de actualizar el estado en memoria. notifyListeners avisa a las pantallas que dependen de AppStore. Las escrituras se ordenan mediante una cola para evitar que dos operaciones consecutivas sobrescriban sus cambios.

Resultado de las pruebas

Se ejecutó flutter test y la suite terminó con cinco pruebas aprobadas. Entre las comprobaciones relevantes para alumnos se incluyen las credenciales, el rango de notas, el límite de aprobación, las operaciones sobre registros y la actualización del correo del perfil. La prueba de interfaz también revisa el acceso al menú y el cierre de sesión.

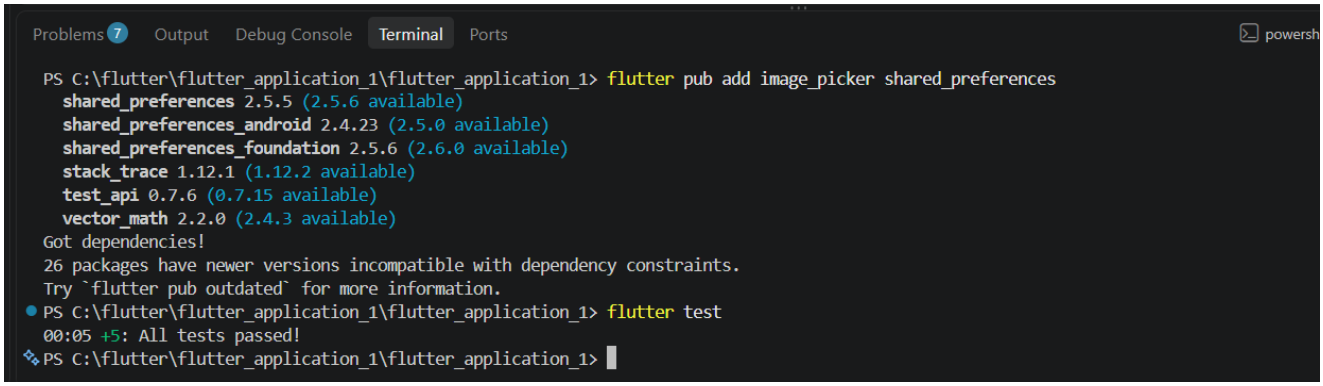


Figura 7. Terminal con el resultado +5 All tests passed después de ejecutar flutter test.

8 Conclusiones

El registro de harold santiváñez con nota 17 aparece en el listado con estado Aprobado. La nota 09 de María López aparece como Desaprobado. Ambos resultados coinciden con la regla de aprobación definida desde 11.

Se comprobó la ejecución de la sección A en Chrome y quedaron documentadas sus cinco pantallas principales. La aplicación utiliza una cuenta de prueba y almacenamiento local; los resultados presentados corresponden a este funcionamiento de demostración.